

CS184/284A Spring 2025 Homework 3 Write-Up

Names: Ruoyu Lin

Link to webpage: <https://cal-cs184-student.github.io/hw-webpages-alxdsptr/> Link to GitHub repository: <https://github.com/cal-cs184-student/hw-webpages-alxdsptr>

Overview

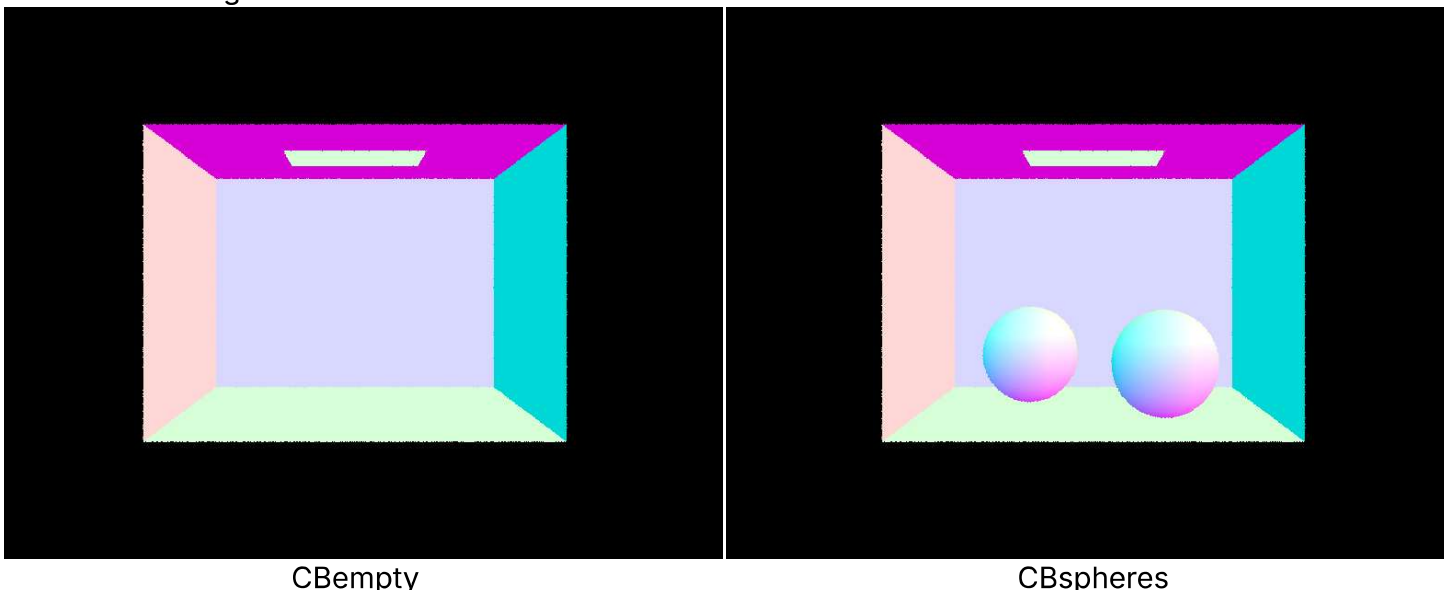
In this homework we implemented a Global Illumination renderer. First, we implemented ray generation as well as ray-triangle and ray-sphere intersections. Then, we implemented the Bounding Volume Hierarchy as an acceleration structure for ray intersection. Building on this, we implemented direct illumination using two different sampling methods, followed by the implementation of global illumination to account for multiple light bounces. Finally, to improve rendering speed, we also implemented adaptive sampling.

Part 1: Ray Generation and Scene Intersection

First we iterate through each pixel in the image space, for each pixel, we generate ns_aa number of random positions within the pixel, and generate rays from the camera origin to these positions (this includes transform the sample point to camera space and then transform the direction vector to screen space). We then iterate through all the primitives in the scene and check for ray-primitive intersections, and find the closest intersection point by updating t_max every time we find a closer intersection. We then compute the normal at the intersection point, and use it to compute the radiance of that sample.

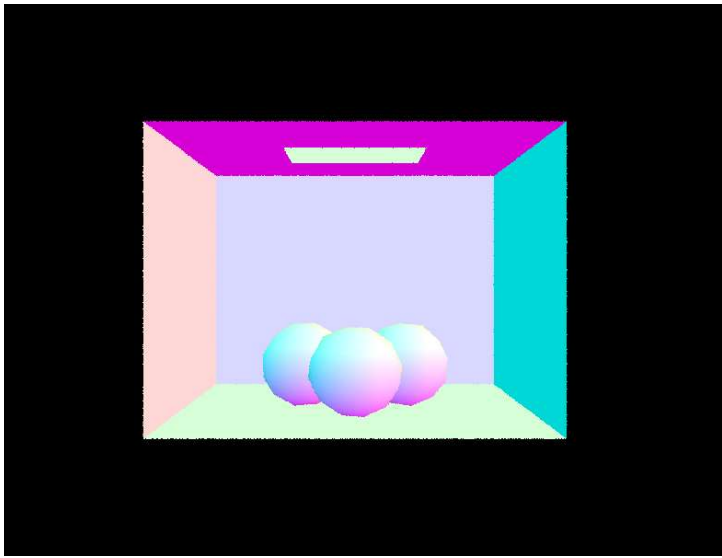
I implemented the moller trumbore algorithm in the slides for ray-triangle intersection. The basic idea is to see triangle as a plane, then solve the linear equation to get the intersection time of the ray and the plane, and then compute the barycentric coordinates of the intersection point to check if the intersection point is within the triangle. And moller trumbore algorithm optimize this process by precomputing some values to avoid redundant calculations. For implementation, I just followed the equations in the slides.

Some rendering results:

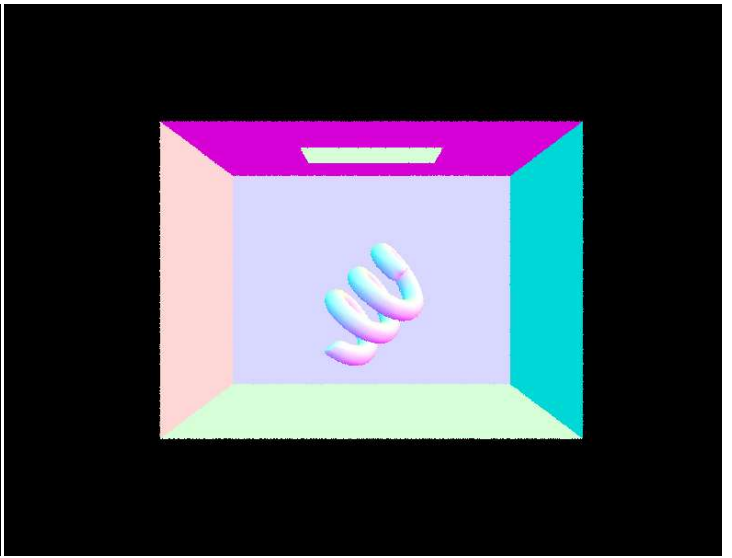


CBempty

CBspheres



CBgems

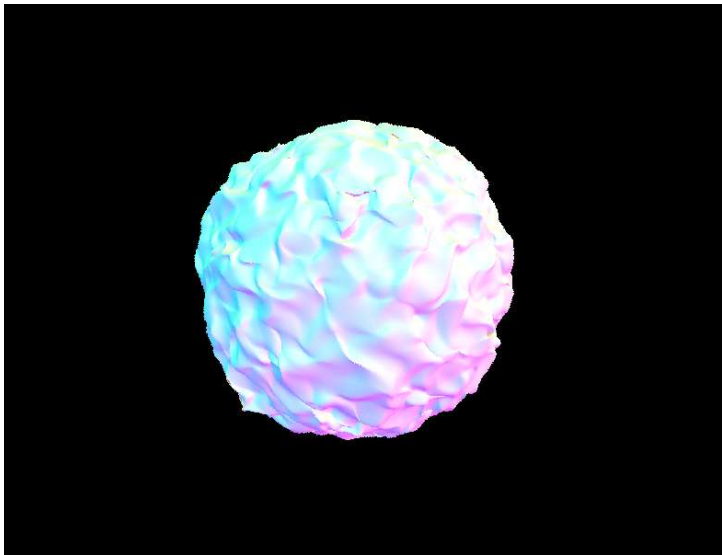


CBcoil

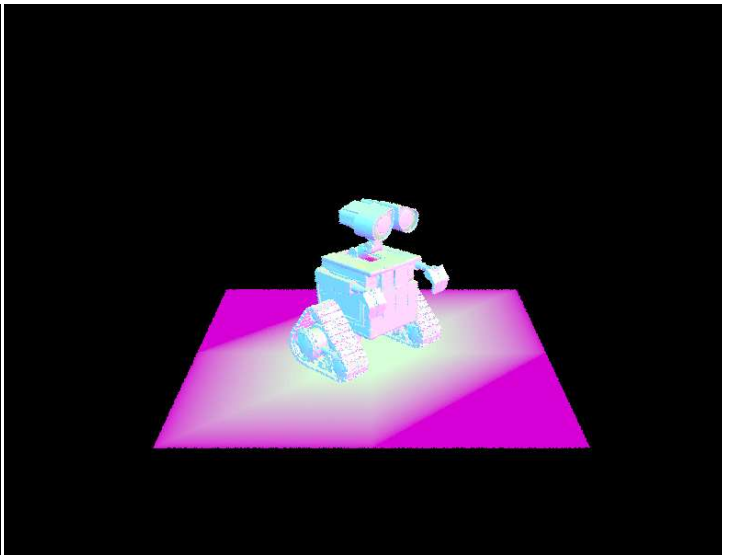
Part 2: Bounding Volume Hierarchy

First, I iterate through each primitive's bounding box to compute the overall bounding box for the current node. If the number of primitives is smaller than `max_leaf_size`, I directly construct a leaf node and return it. Otherwise, I select the axis with the largest extent to perform the split. To choose the splitting point, I calculate the median of all bounding box centroids along this axis. The advantage of this approach is that it guarantees the resulting BVH tree is perfectly balanced with a depth of exactly $\log n$, while also ensuring that no node remains empty. Finally, I partition the primitives into two halves based on this splitting point and recursively call the `construct_bvh` function.

Below is the rendering results of two large dae files:



blob



wall-e

Below is the rendering time comparison between ray tracing with and without BVH for two moderately complex dae files:

mesh	without BVH	with BVH
CBcoil	195s	1.23s
cow	45.64s	1.22s

From the results above, we can see that the BVH significantly improves the performance of ray tracing. This is because the BVH reduces the time complexity of ray-primitive intersection from $O(n)$

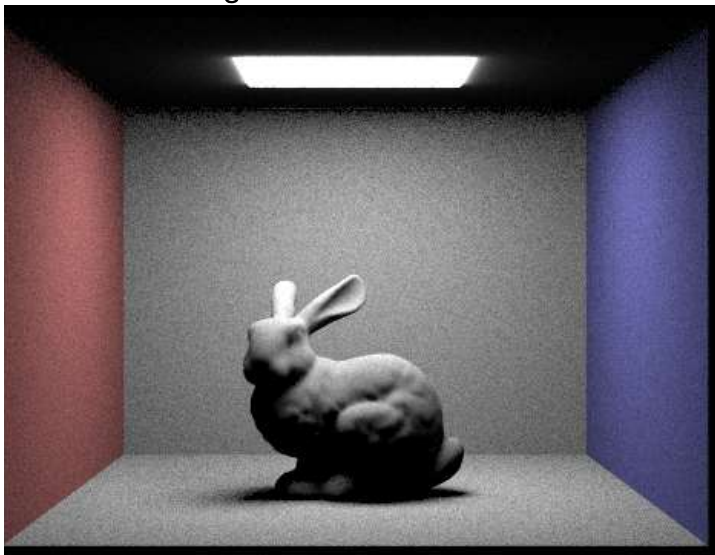
to $O(\log n)$. With BVH, we can quickly eliminate large groups of primitives that do not intersect with the ray, which leads to a significant reduction in the number of intersection tests needed. Moreover, computing the intersection with bounding boxes is much cheaper than computing the ray-triangle intersection.

Part 3: Direct Illumination

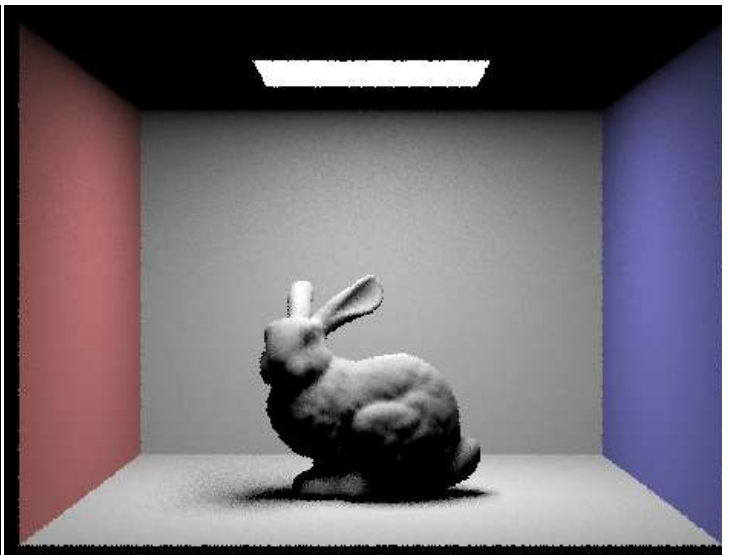
For hemisphere sampling, first we compute the number of samples as the number of lights times the number of samples per light to make the results comparable to importance sampling. Then for each sample, we randomly sample a direction using hemisphereSampler, transform it to world space, and then cast a ray in that direction to check if it intersects with any light. If it does, we compute the radiance of that point using the rendering equation $f_r(w_i, w_o) L_i(w_i) \frac{(n \cdot w_i)}{p(w_i)}$, in which L_i is the emission of the light source. And the pdf for every sample is the same, which is $\frac{1}{2\pi}$ since we are sampling uniformly on the hemisphere.

For importance sampling, we iterate through all the light sources, then for each area light, we compute ns_area_light number of samples by randomly sampling a point on the area light source, and then cast a ray from the intersection point to the sampled point to check if it intersects with any object. If it doesn't, we compute the radiance of that point using the same rendering equation as above. The pdf now depends on the point we sampled, but the sample_L function helps us to compute that.

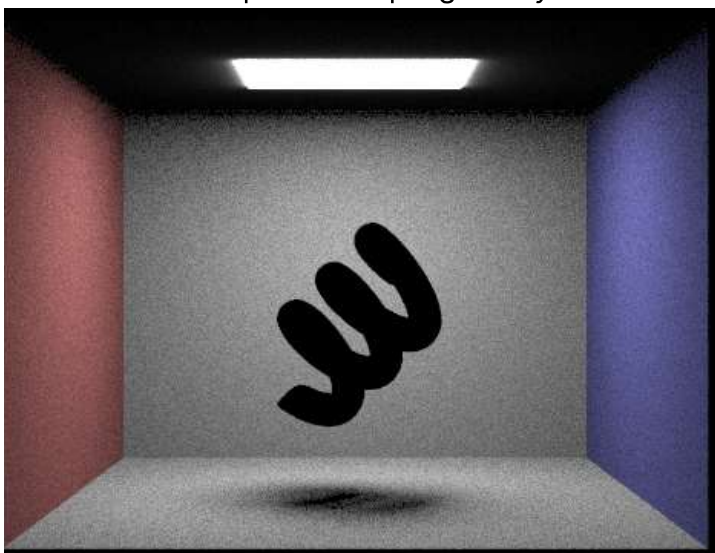
Some rendering results:



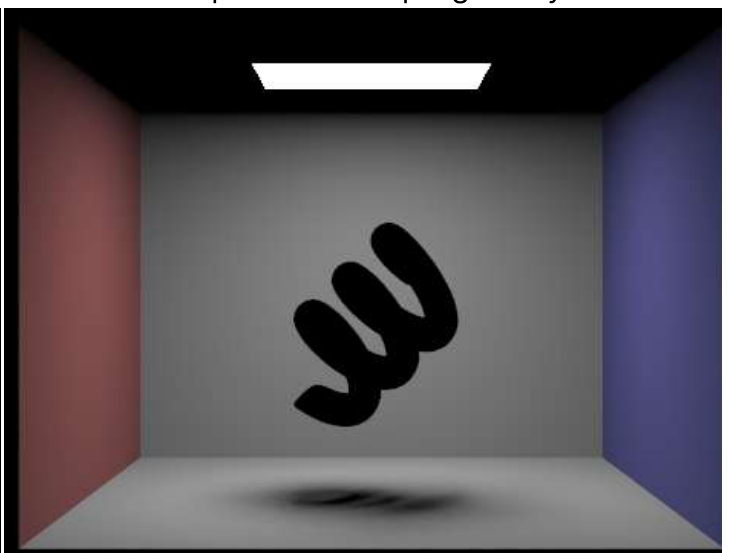
hemisphere sampling bunny



importance sampling bunny

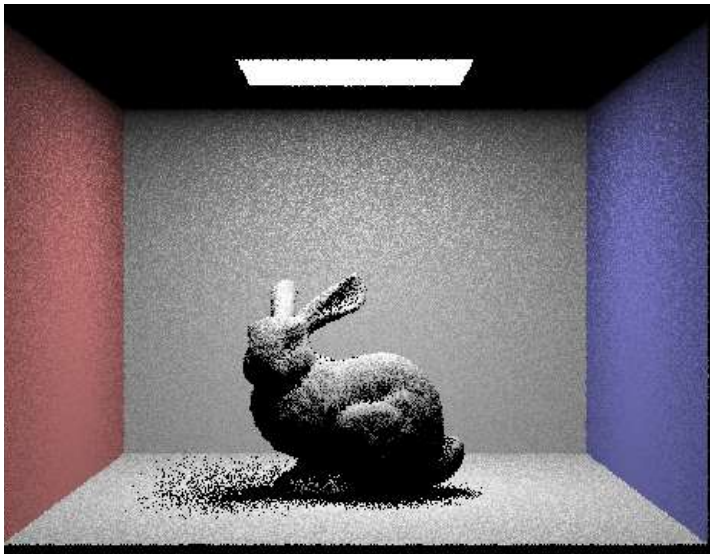


hemisphere sampling coil

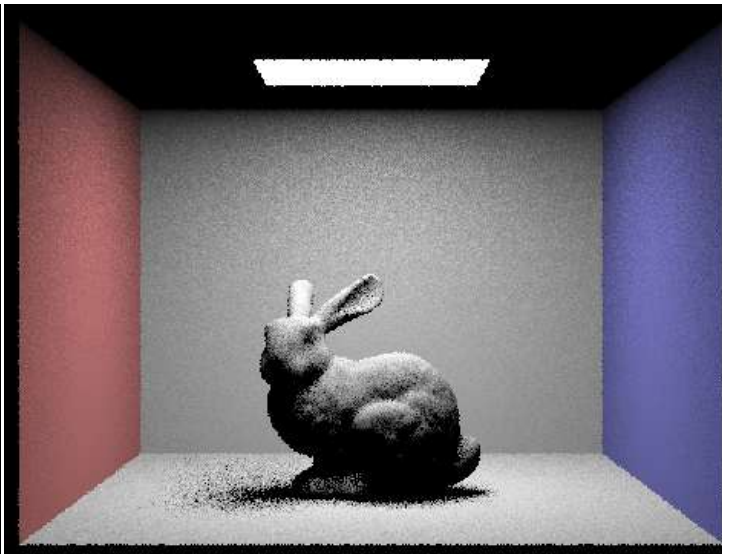


importance sampling coil

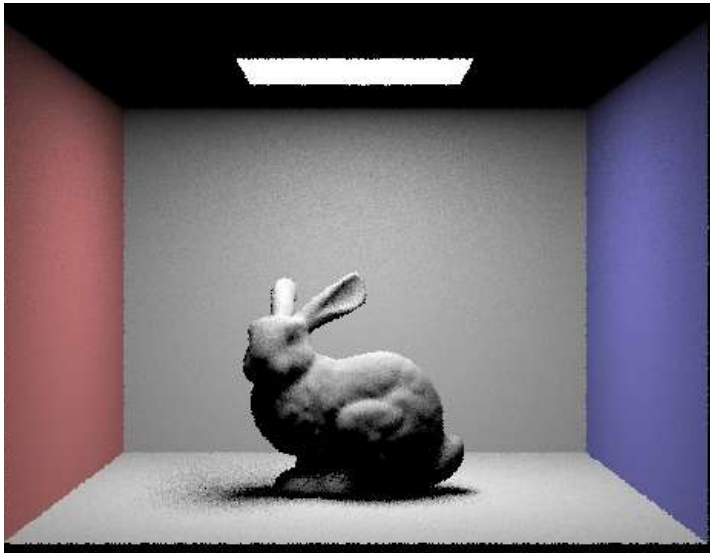
Comparisons of different numbers of light rays: we can see that the more light rays we use, the less noise there will be in the soft shadow areas



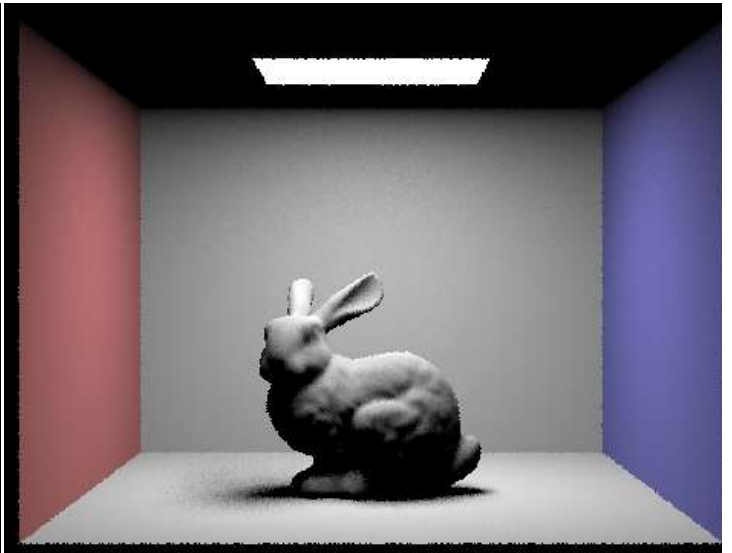
1 light ray



4 light rays



16 light rays



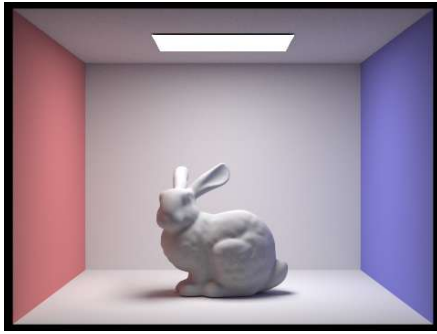
64 light rays

Compared to uniform hemisphere sampling, importance sampling produces higher-quality images with less noise under the same sample count. In hemisphere sampling, a large portion of the sampled rays fail to hit any light source, resulting in zero radiance contributions and effectively 'wasting' those samples. Conversely, importance sampling directs rays specifically toward light sources, ensuring a more efficient utilization of each sample to capture meaningful radiance.

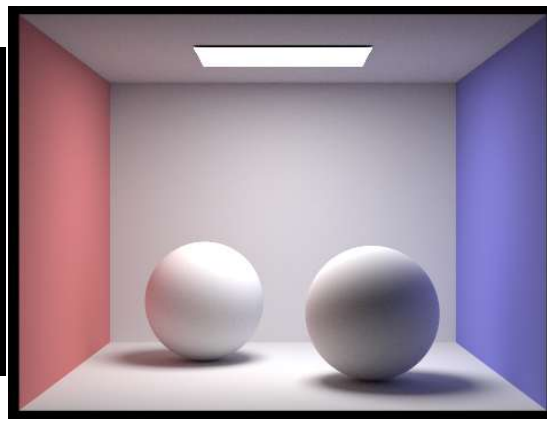
Part 4: Global Illumination

Inside the `at_least_one_bounce_radiance` function, I first check if the depth of the ray is smaller or equal to 0, if it is, I immediately return 0. Then if the depth of the ray equals to 1, I compute the `one_bounce_radiance` and return it. Otherwise, if `isAccumBounces` is set to true, I compute the `one_bounce_radiance`, and add it to the result variable. Then to implement Russian Roulette, I use a stop probability of 0.35, if the ray's depth is smaller than `max_ray_depth`, the function will immediately return at 35% probability. Then I randomly sample a direction using `sample_f`, and cast a ray in that direction to check if it intersects with any object. If it does, I recursively call `at_least_one_bounce_radiance` with the new ray and depth minus 1 to compute the $L_i(w_i)$ term in the rendering equation, and then compute the radiance of that point using the same rendering equation as above, and add it to the result variable. Finally, I return the result variable.

Some rendered images with global illumination:



CBbunny

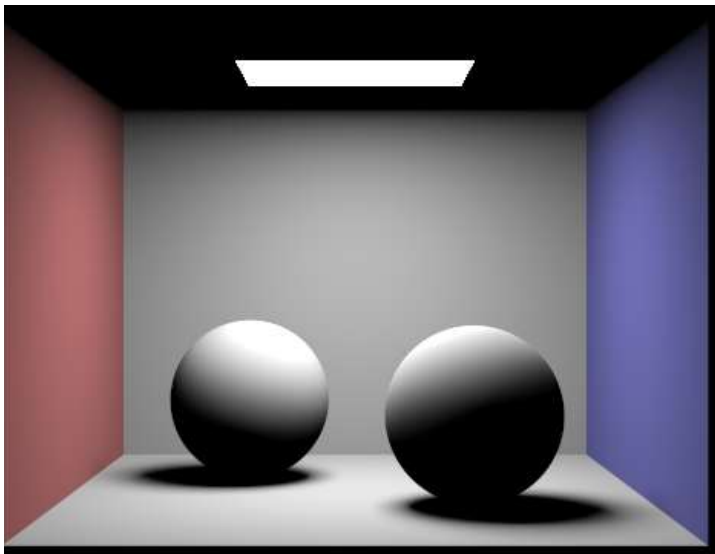


CBspheres_lambertian

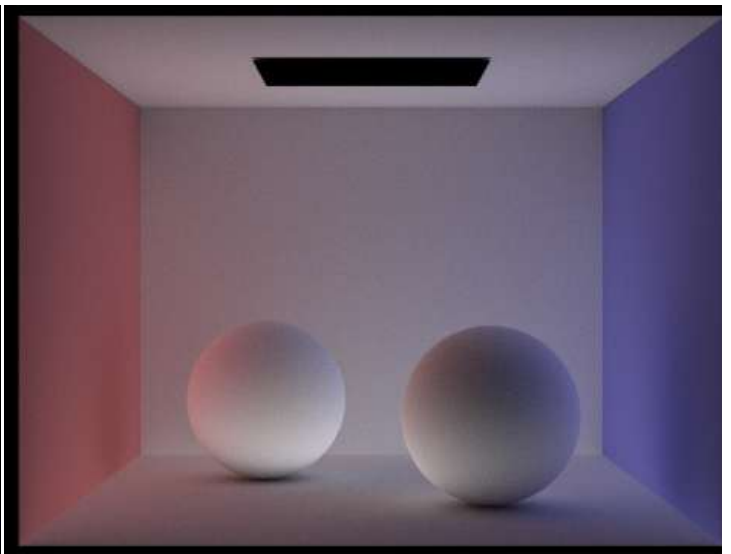


bunny

Comparison of only direct illumination and only indirect illumination on CBspheres_lambertian.dae:



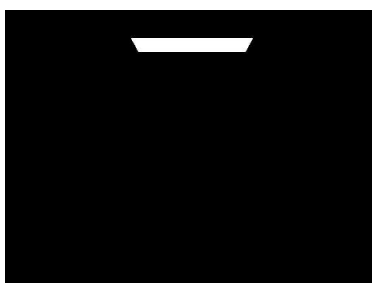
only direct illumination



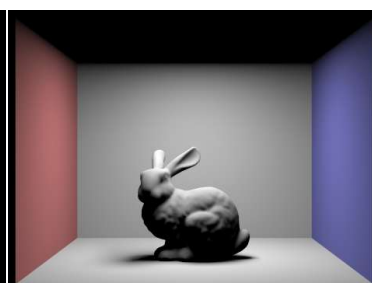
only indirect illumination

Comparison of accumulated and unaccumulated bounces:

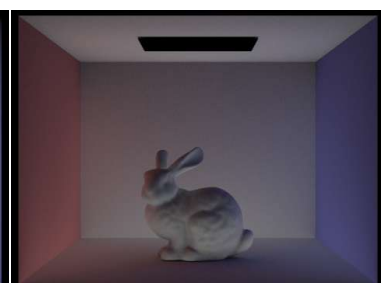
In the second and third bounces, we can observe that the left side of the floor and the rabbit are tinted with a subtle red hue due to reflections from the left wall. Similarly, the right side of the floor and the rabbit are tinged with a faint blue from the right wall's reflections. At the same time, the ceiling is illuminated by light bouncing off the walls and floor, and the lower half of the rabbit is lit by reflections from the ground. These multiple bounces significantly enhance the realism of the final render. In contrast, if rasterization were used, the ceiling would remain pitch black, which does not align with how light behaves in the real world.



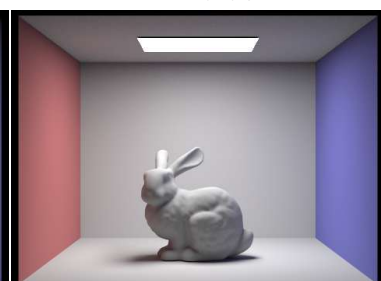
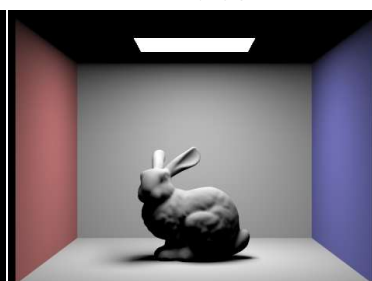
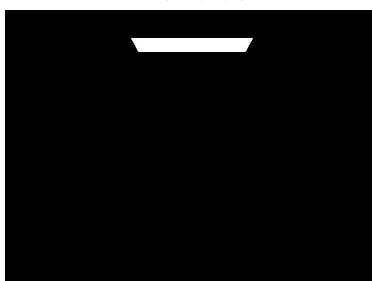
m0 false

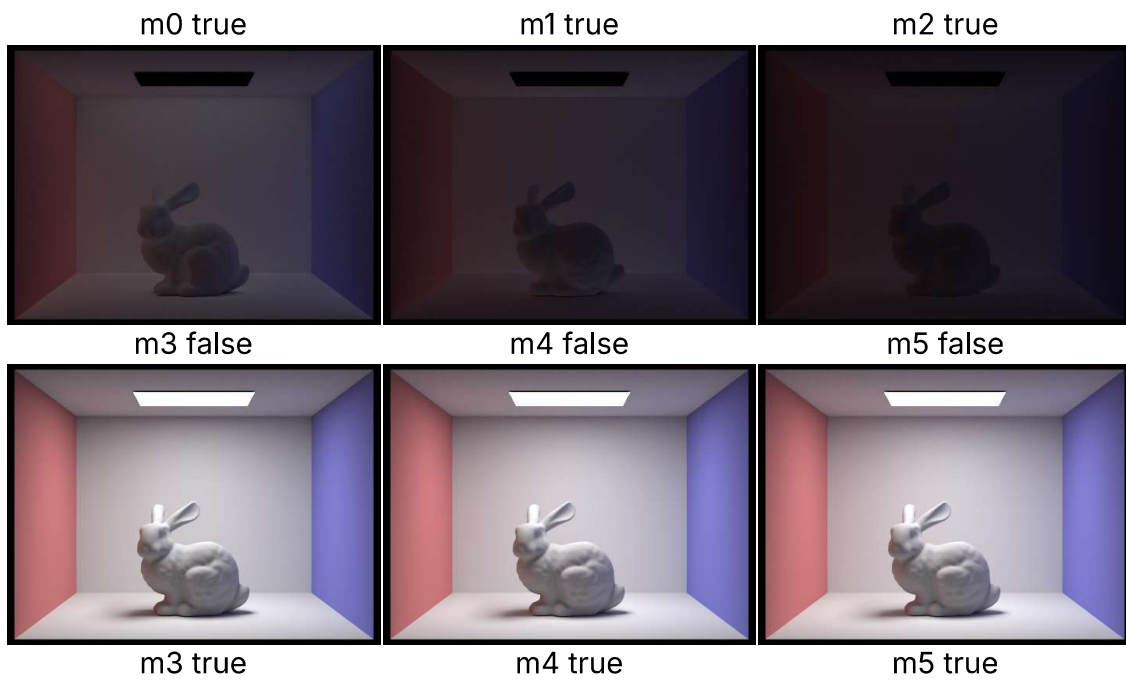


m1 false

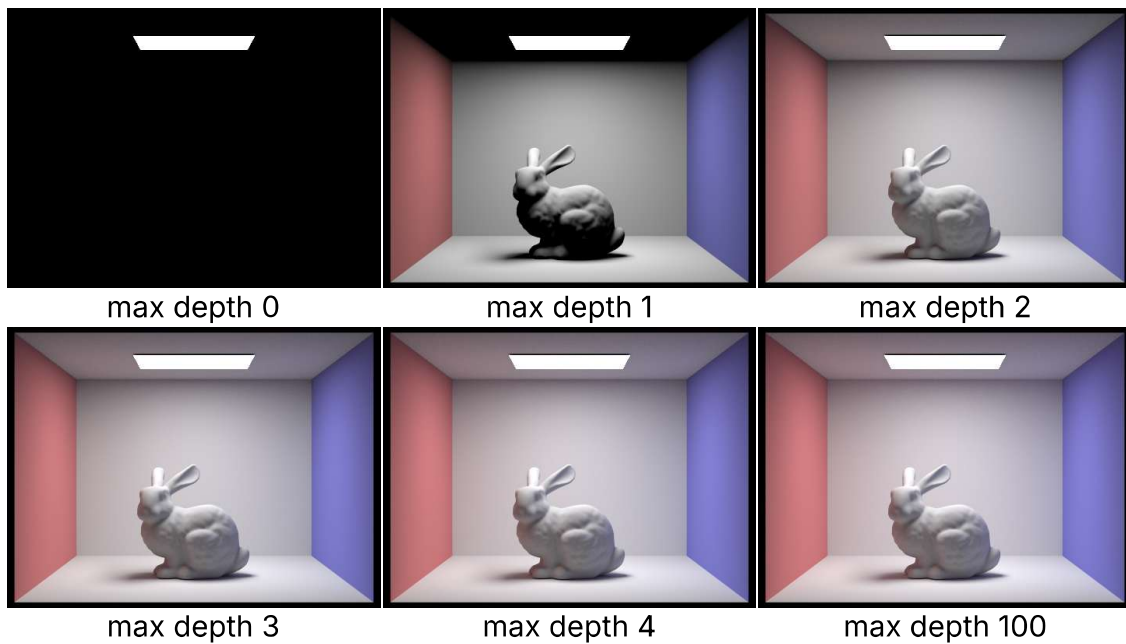


m2 false

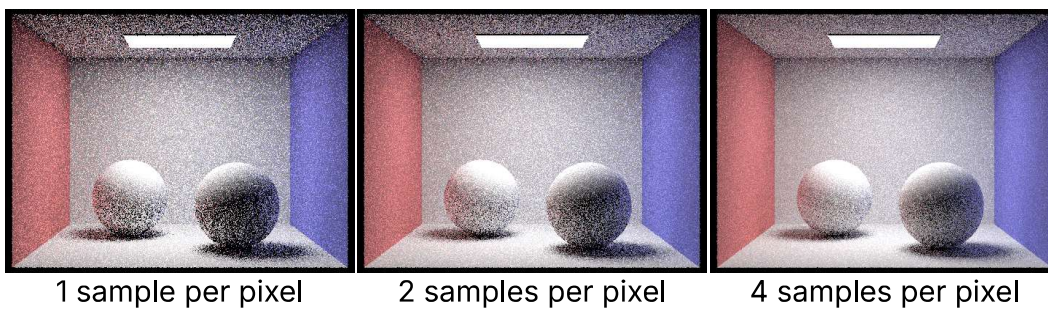


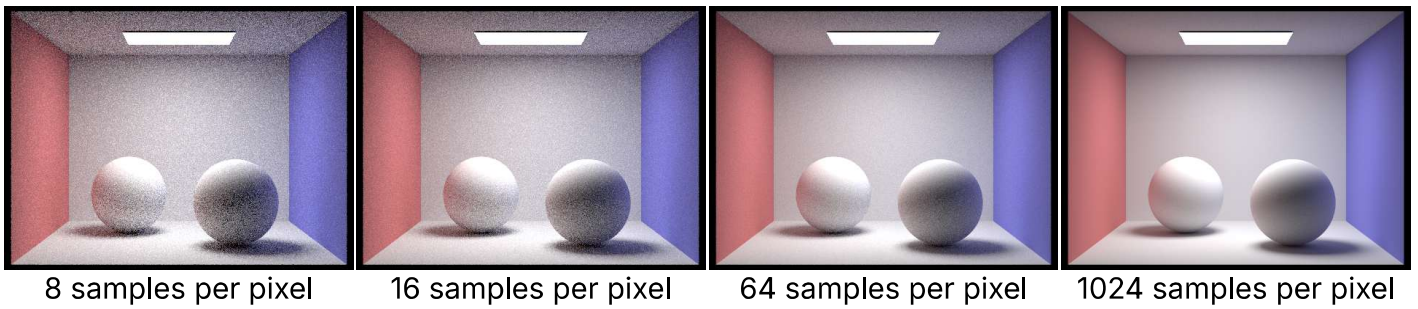


Russian Roulette rendering results:



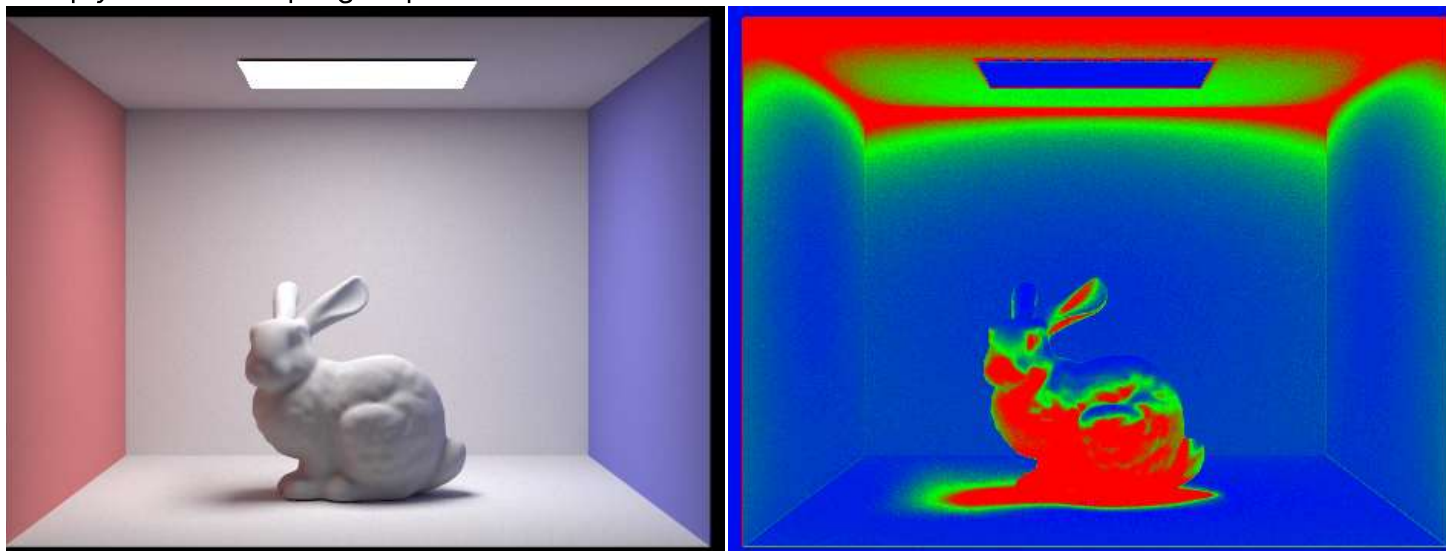
CBspheres_lambertian.dae with various sample-per-pixel rates: we can see that the more samples per pixel, the less noise there is in the final image.





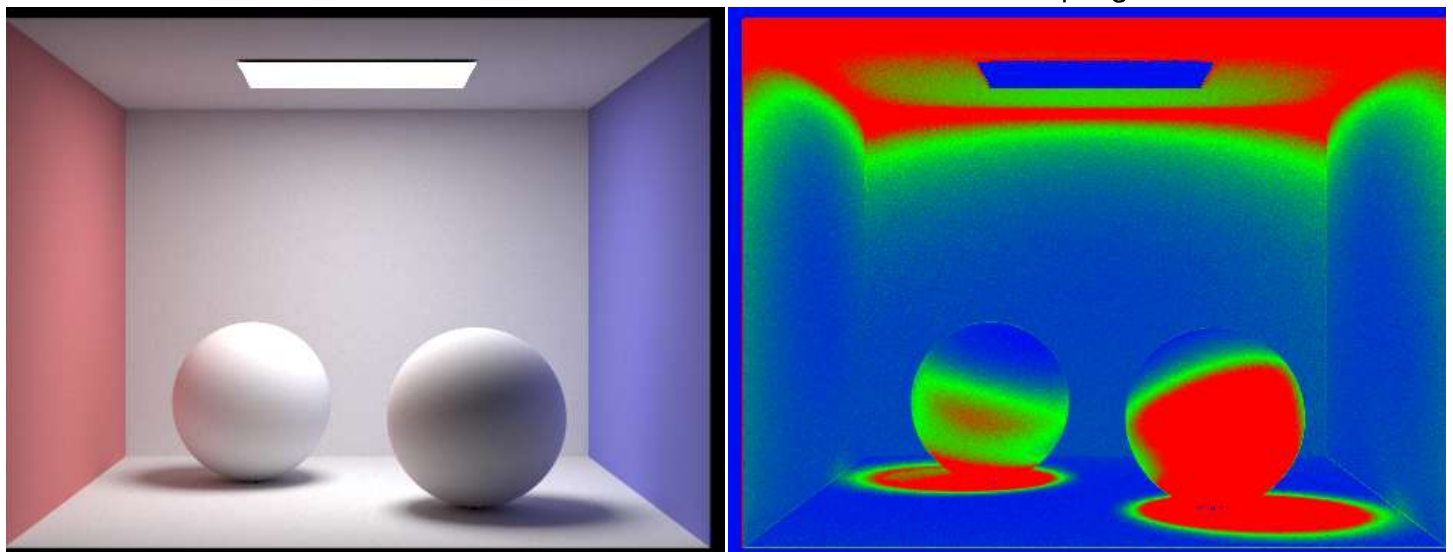
Part 5: Adaptive Sampling

The idea of adaptive sampling is that if the change of the pixel's value is small, which means that the pixel is already converged, we can stop sampling more rays for that pixel to save time. And the method to determine if it's converged here is to compute a value proportional to the variance divided by the square root of the number of samples, and check if it's smaller than a threshold proportional to the mean radiance. In my implementation, I maintain a running sum of the illum and the sum of squares of the illum for the pixel, and then compute the mean and variance using these two values. Then I compute the V as $1.96 \sqrt{\frac{\sigma}{\sqrt{n}}}$ and check if it's smaller than $maxTolerance * \mu$. If it is, I just simply exit the sampling loop.



rendered results

Sampling rate



Rendered results

Sampling rate

(Optional) Part 6: Extra Credit Opportunities

surface area heuristic

Model Name	Primitives	Surface Area Heuristic (SAH)		Median Split		Speedup (Render)
		BVH Build Time	Render Time	BVH Build Time	Render Time	
CBspheres_lambertian	14	0s	39.0318s	0s	48.8743s	1.252166
CBbunny	28,588	0.0685s	47.4158s	0.0409s	78.5748s	1.657144
CBdragon	100,012	0.3022s	48.7255s	0.1813s	69.3703s	1.423696

We can see that the surface area heuristic only increases the BVH construction time by a small amount, but it significantly reduces the rendering time, achieving an average speedup of 1.44x. And generally, the more complex the model is, the more improvement we can get from using SAH.